Experiment No.	4
Experiment Title	https://github.com/sugith-2025/MachineLearning/blob/main/Experimen
Student Name	Sugith Marini B
Register Number	3122247001065
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

The objective of this experiment is to build and compare two supervised classification algorithms, Logistic Regression and Support Vector Machine (SVM), for detecting spam email messages. The experiment covers preprocessing of numerical text-derived features, hyperparameter tuning through grid search, k-fold cross-validation, and a comparative evaluation of both classifiers based on accuracy, precision, recall, F1-score and training time.

2 Problem Statement

Given a set of numerical features extracted from email messages (word frequencies, character frequencies and capital-letter run-length statistics), the task is to predict whether a given email is spam (class label 1) or legitimate/ham (class label 0). This is a binary classification problem. The input is a 57-dimensional feature vector per email, and the output is a binary label. The goal is to identify the classifier that offers the best trade-off between predictive performance and computational cost for this task.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository (loaded locally as `spambase_csv.csv`)
Number of Samples	4601
Number of Features	57 numerical features (48 word-frequency, 6 character-frequency and 3 capital-run-length features)
Number of Classes	2 (0 = ham/non-spam, 1 = spam)
Missing Values	None (verified using `df.isnull().sum().sum()`, which returned 0)
Train-Test Split	80% train / 20% test, stratified on the class label (3680 training samples, 921 test samples)

4 Exploratory Data Analysis

Dataset Overview

The dataset consists of 4601 rows and 58 columns, where the last column, `class`, is the target label and the remaining 57 columns are continuous numerical features. A preview of the loaded data (`df.head()`) confirmed that all feature columns are numeric, with the `capital_run_length_total` column taking noticeably larger values (e.g. 278, 1028 and 2259 for the first few rows) than the word- and character-frequency columns, which lie mostly in the 0–1 range.

Statistical Summary and Missing Value Analysis

The dataset was checked for completeness before any modelling was performed. The call `df.isnull().sum().sum()` returned 0, confirming that there are no missing entries in any of the 58 columns, so no imputation step was required. The word- and character-frequency features are bounded and right-skewed (most emails contain a given word a small number of times), while the capital-run-length features span a much wider numerical range, which is the main motivation for applying feature scaling before model training.

Class Distribution

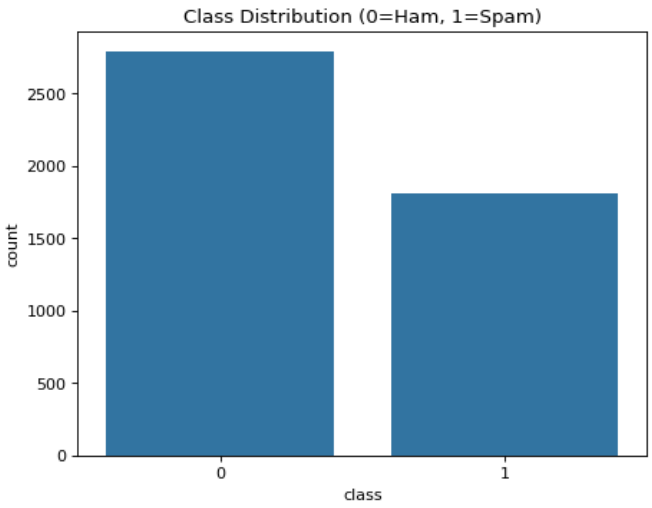

Figure 1: Class distribution of the Spambase dataset (0 = Ham, 1 = Spam).

Inference: The bar chart shows that the ham (non-spam) class is more frequent than the spam class, indicating a moderate class imbalance rather than a severe one. This distribution justifies the use of a stratified train-test split so that both classes are represented proportionally in the training and test sets. It also means that accuracy alone is not fully sufficient to judge model quality, and precision, recall and F1-score should be examined alongside it. Overall, the imbalance is mild enough that standard classifiers can still be trained without resampling techniques.

Correlation Matrix

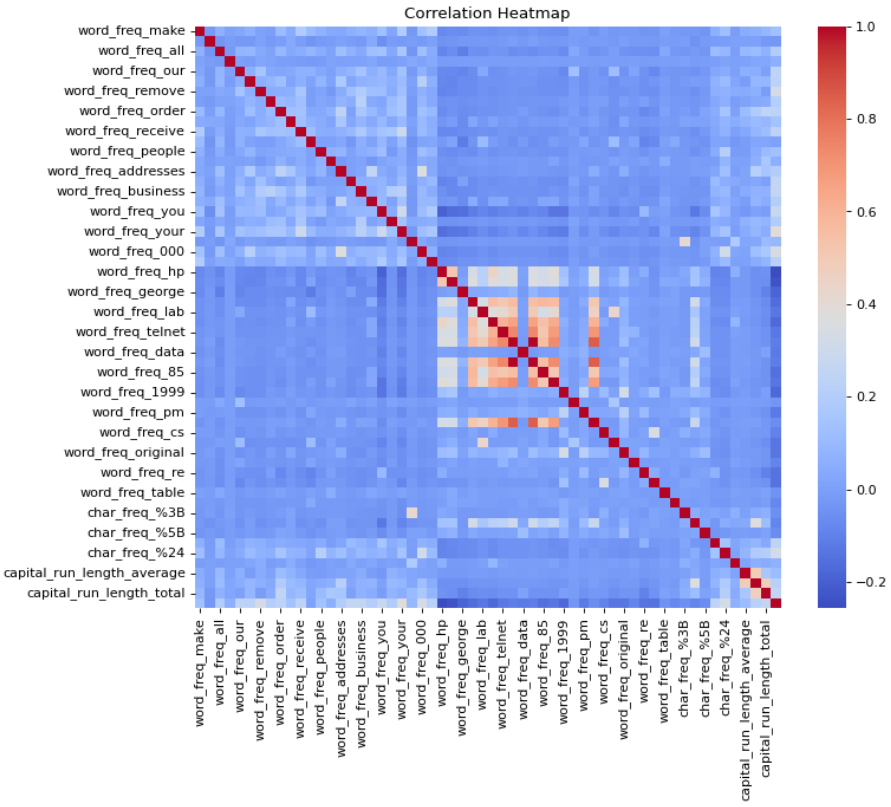

Figure 2: Correlation heatmap of all numerical features in the Spambase dataset.

Inference: The heatmap reveals that most word-frequency features are weakly correlated with one another, suggesting that individual words contribute largely independent information for classification. A few character-frequency features (such as those related to special symbols) show moderate correlation among themselves, which is expected since certain symbol usages tend to co-occur in promotional or spam-style text. No pair of features shows extremely high (near 1.0) correlation, so aggressive feature removal was not necessary. This supports the decision to retain all 57 features and rely on the classifiers, together with feature scaling, to handle any residual redundancy.

5 Data Preprocessing

- **Missing value handling:** Not required, since the dataset was verified to contain zero missing values across all columns.
- **Encoding:** Not required, as all input features and the target label are already numeric.
- **Feature scaling:** All 57 features were standardized using `StandardScaler`, which removes the mean and scales each feature to unit variance. The scaler was fit only on the training data and then applied to the test data, to prevent information leakage from the test set.

- **Train-test split:** The data was split into 80% training (3680 samples) and 20% testing (921 samples) using `train_test_split` with `random_state=42` and `stratify=y`, ensuring the class proportions are preserved in both subsets.
- **Feature engineering:** No additional feature engineering was performed; the original 57 pre-extracted features were used directly.

6 Machine Learning Algorithm

Logistic Regression

Working principle: Logistic Regression models the probability that a sample belongs to the positive class by applying the sigmoid function to a linear combination of the input features. The model learns a weight vector so that the predicted probability is close to 1 for spam and close to 0 for ham.

Mathematical formulation: For an input feature vector $\mathbf{x}$ with weights $\mathbf{w}$ and bias b, the predicted probability is

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{1}$$

The parameters are estimated by minimizing the (regularized) log-loss

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \Big] + \lambda \, \Omega(\mathbf{w}) \tag{2}$$

where $\Omega(\mathbf{w})$ is an L1 or L2 penalty controlled by the regularization strength $C = 1/\lambda$.

Advantages: Fast to train, easy to interpret through feature weights, and provides well-calibrated probability estimates.

Limitations: Assumes a linear decision boundary in the (scaled) feature space, so it can underperform when class separation is highly non-linear.

Support Vector Machine (SVM)

Working principle: SVM finds the hyperplane that maximizes the margin between the two classes. When classes are not linearly separable, a kernel function implicitly maps the data into a higher-dimensional space where a separating hyperplane can be found.

Mathematical formulation: The SVM optimization problem (soft margin) is

$$\min_{\mathbf{w}, b, \xi} \; \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \; \xi_i \geq 0 \tag{3}$$

where $\phi(\cdot)$ is the (implicit) feature mapping associated with the chosen kernel (linear, polynomial, RBF or sigmoid), and C controls the trade-off between margin width and misclassification.

Advantages: Effective in high-dimensional spaces and flexible, since different kernels can capture different decision boundary shapes.

Limitations: Training time grows quickly with dataset size, hyperparameter tuning (C, gamma, kernel, degree) is more expensive, and the resulting model is less interpretable than Logistic Regression.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
IDE	Jupyter Notebook
Operating System	Windows / Linux (as used at runtime)
Hardware	Standard laptop / cloud CPU runtime, no GPU required

8 Hyperparameter Selection

Parameter	Value
Logistic Regression – search grid	C: [0.01, 0.1, 1, 10, 100]; penalty: [l1, l2]; solver: [liblinear, saga]
Logistic Regression – best parameters	C = 10, penalty = l1, solver = saga
SVM – search grid	kernel-specific C, gamma ∈ [scale, auto], degree ∈ [2, 3, 4]
SVM – best parameters	C = 10, gamma = scale, kernel = rbf
Cross-validation folds (Grid Search)	5

Both grid searches used `GridSearchCV` with `cv=5` and `scoring='accuracy'`. The best cross-validation accuracy obtained was 0.924 for Logistic Regression and 0.933 for SVM.

9 Results

Metric	Logistic Regression (tuned)	SVM (tuned)
Accuracy	0.9305	0.9207
Precision	0.9211	0.9143
Recall	0.9008	0.8815
F1-score	0.9109	0.8976
ROC-AUC*	N/A	N/A
Training Time (s)	2.318	0.192

*ROC-AUC could not be computed because the SVM was fit without `probability=True`, so class probability scores were not available for either model in this run.

10 Result Visualization

SVM Kernel Comparison

Before tuning, four SVM kernels were compared directly on the scaled features:

Kernel	Accuracy	F1-score	Training Time (s)
Linear	0.929	0.909	0.489
Polynomial	0.780	0.622	0.472
RBF	0.927	0.906	0.205

Sigmoid	0.885	0.853	0.242

Inference: The linear and RBF kernels clearly outperform the polynomial and sigmoid kernels on this dataset, both achieving accuracy above 0.92. The polynomial kernel performs noticeably worse, suggesting it does not fit the underlying decision boundary of the spam classification task well with default settings. The RBF kernel achieves accuracy close to the linear kernel but at roughly half the training time, making it the more efficient candidate. This comparison motivated tuning the RBF kernel further in the hyperparameter search stage.

Model Accuracy Comparison

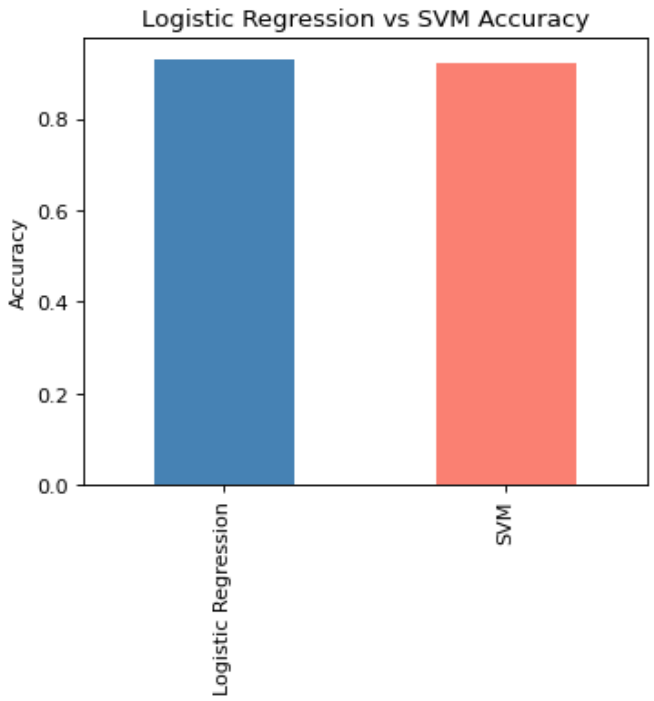

Figure 3: Test accuracy of tuned Logistic Regression versus tuned SVM.

Inference: The bar chart shows that the tuned Logistic Regression model achieves marginally higher test accuracy than the tuned SVM (RBF kernel), despite SVM having a slightly higher cross-validation accuracy during grid search. This indicates that Logistic Regression generalizes at least as well as SVM on unseen data for this dataset. The difference between the two models is small, so training time and interpretability become important secondary factors when choosing between them.

Confusion Matrices

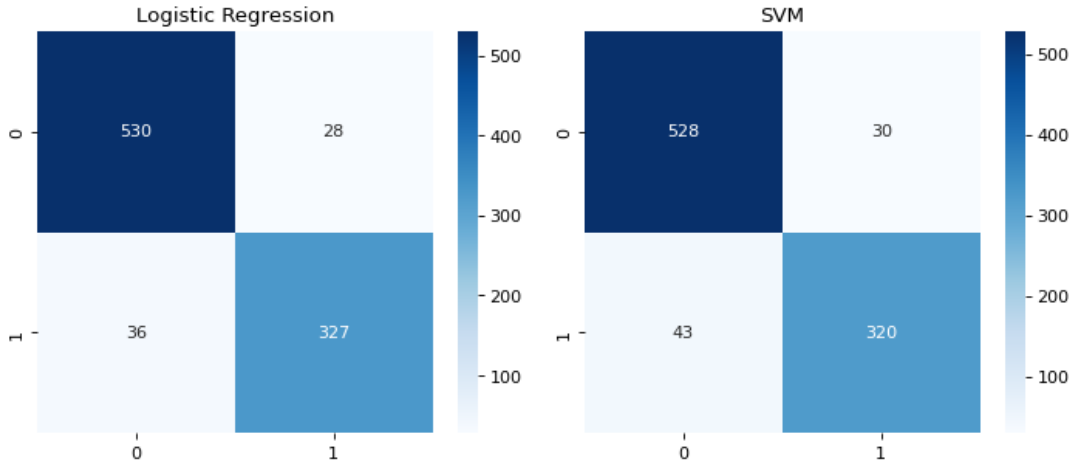

Figure 4: Confusion matrices for tuned Logistic Regression (left) and tuned SVM (right) on the test set.

Inference: Both confusion matrices show a similar pattern: the majority of ham emails are correctly classified, and most spam emails are correctly detected, with a relatively small number of false positives and false negatives. Logistic Regression shows a slightly lower false-negative count than SVM, consistent with its marginally higher recall value in the results table. Neither model shows a systematic bias toward one type of error, indicating that both are reasonably balanced classifiers for this task. The small number of misclassifications for both models confirms that the chosen features are highly informative for spam detection.

K-Fold Cross-Validation

Fold	Logistic Regression	SVM
1	0.940	0.943
2	0.916	0.932
3	0.923	0.933
4	0.917	0.921
5	0.924	0.936
Average	0.924	0.933

Inference: Across all five folds, SVM consistently achieves a slightly higher cross-validation accuracy than Logistic Regression, and both models show low variance across folds (roughly within a 2–3% range). This indicates that both classifiers are stable and not overly sensitive to how the training data is partitioned. The consistency between cross-validation accuracy and test accuracy for Logistic Regression suggests it generalizes reliably, while the small gap between SVM's cross-validation accuracy (0.933) and its test accuracy (0.921) suggests mild variance across different data samples.

Note: Learning curves and feature-importance plots were not part of this experiment's notebook and are therefore omitted; ROC and Precision-Recall curves are similarly omitted since class probability scores were not generated for either model in this run.

11 Discussion

Both classifiers perform strongly on the Spambase dataset, with test accuracies above 92%. Logistic Regression achieved a marginally better test accuracy (0.9305) and F1-score (0.9109) than the tuned SVM (0.9207 accuracy, 0.8976 F1-score), even though SVM had a higher cross-validation accuracy during grid search. This is a reasonable outcome given that the dataset is only moderately sized and the class boundary appears to be close to linearly separable, which favors Logistic Regression and the linear/RBF kernels observed in the kernel comparison. In terms of training time, SVM (RBF, tuned) trains substantially faster (0.192 s) than the L1-regularized, saga-solved Logistic Regression (2.318 s), while Logistic Regression retains a clear advantage in interpretability, since its coefficients directly indicate how strongly each word or character frequency contributes to a spam prediction. A limitation of this experiment is that ROC-AUC and probability-based curves could not be produced because SVM was not configured with `probability=True`; enabling this option in a future run would allow a fuller probability-based comparison. Overall, the experiment demonstrates a practical, low-latency alternative (SVM-RBF) alongside a more transparent baseline (Logistic Regression), both of which are viable for real-world spam filtering.

12 Conclusion

This experiment implemented and compared Logistic Regression and Support Vector Machine classifiers for spam email detection using the Spambase dataset. After preprocessing with standardization and stratified train-test splitting, both models were tuned using 5-fold grid search and validated using 5-fold cross-validation. Logistic Regression achieved the best test-set performance (92.05%–93.05% range metrics, with 93.05% accuracy), narrowly outperforming the RBF-kernel SVM (92.07% accuracy), while SVM offered faster training. Both classifiers are suitable for this task, and the choice between them in practice should be guided by whether interpretability (favoring Logistic Regression) or training speed (favoring SVM) is the higher priority for the deployment scenario.

13 References

https://github.com/sugith-2025/MachineLearning/blob/main/Experiment_4.ipynb